

Document Version: 1.0 (a newer version number means an update on the project)
Assignment Date: January 23, 2025, 5:30pm
Due Date: **February 11, 2025, 11:59pm**

Objectives:

- Get familiar with Go.
- Get familiar with AWS.

PART 1 (50 PTS): Clock Server

Write a concurrent `clockserver.go` program, similar to `clock2.go` covered in Chapter 8 of our reference Go book (Donovan & Kernighan). Accept a port number as a command line argument on which to listen for client requests. Report the server's current system time to all clients concurrently once per second. In addition, write a program called `clockwall.go`, similar to `netcat1.go` covered in the book, which acts as a client of multiple clock servers at once. Read the time from each clock server and print the results to the screen, akin to wall clocks seen in some business offices. Your wall clock program must accept a list of clock server machine information in the form of `ip-address:port-number` pairs as command line arguments.

Run two separate EC2 instances, one with the system clock set to UTC time (the default) and the other to a local time zone of your choice (specify in report). Use the following command in Ubuntu to reconfigure the time zone:

```
$ sudo dpkg-reconfigure tzdata
```

Run a clock server on each EC2 instance and run the client on your personal machine. (Hint: Check security settings.) Observe and document time zone differences. Document your configurations and observations with supporting **screenshots**.

Notes on Go

Pay close attention to the documentation for `net.Listen`. The second argument is an address string with format "host:port". DO NOT use a host - not even `localhost` - just leave it blank and specify only the port number, like `":8080"`. Using a host will listen only on the network adapter bound to that host name. So `"localhost:8080"` will listen for incoming requests from the loopback adapter (`lo`, `127.0.0.1`), in which case you could connect only from yourself (not what you want). Leaving it blank, it will accept from *any* adapter, such as the one connected to the outside world (what you want).

`WaitGroup` from the `sync` package will be useful for our projects. I suggest that you read the related section from the reference Go book (pages 237-238) and/or look it up for more information about it.

PART 2 (50 PTS): Concurrency & Parallelism with Goroutines

Read about *goroutines* from the Go book (Donovan & Kernighan). In your report, briefly summarize the key differences between goroutines, kernel threads, and processes. What is the difference between concurrency and parallelism?

Take the 3D surface computation code from Section 3.2 of the Go book as an existing CPU-bound **sequential** program and call it `surface_s.go`. First, remove all I/O operations, such as `Printfs`, to make sure that it is purely CPU bound. Next, increase the problem size (number of iterations of the loop that you will later parallelize, i.e. `cells`) to 3000 so that the execution time is more substantial. Finally, measure the execution time of the sequential program using the `time` shell command and record the reported `real` (wall clock) value.

In addition, write a **parallel** version called `surface_p.go`, in which you execute the main loop concurrently using goroutines. Measure how much faster it runs on a multiprocessor `t2.2xlarge` instance in AWS.

While developing your code locally on your own computer, first determine how many CPUs your computer has (check `nproc` and `cat /proc/cpuinfo`) and find the optimal number of goroutines to use for your machine. Read about GOMAXPROCS in Section 9.8.3 in detail and measure the performance of your parallel program versus various values of GOMAXPROCS. Finally, determine the optimal GOMAXPROCS value for the `t2.2xlarge` instance and measure its performance in AWS.

Write your `surface_p.go` in a flexible way so that the number of goroutines is passed as a command line argument. Make sure that the parallel version performs exactly the same calculation as the sequential version. (For debugging purposes, you can temporarily print their SVG image outputs and see if they match. Since the parallel version will print lines nondeterministically, you can sort the lines of output before comparing.) Finally, analyze the *speed-up* that the parallel version achieves (execution time of sequential version divided by the execution time of parallel version) for various numbers of goroutines: starting from two, in increments of two, until twice the number of cores or virtual processors for `t2.2xlarge`. For more reliable results, repeat each experiment multiple times (minimum three replicates) and average the results of different runs for each case. Clear your caches before each execution in order to eliminate any effect of caching. Use the following shell command to clear your caches in Linux. In a shell as superuser (root):

```
# sync && echo 3 > /proc/sys/vm/drop_caches
```

If using `sudo` in a shell as a regular user, the redirection must also be performed as `sudo` and so it requires special attention:

```
$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
```

Briefly mention your experimental method in your report. **Plot your achieved *speed-up* values on a graph** where the x -axis shows the number of goroutines and the y -axis shows the achieved speed-up. Analyze your results.

Collaboration and Code Sharing Policy

This course assumes that assignments submitted by students will be generated by the students themselves working individually. This policy indicates the following constitute violations of academic honesty: a student has another person/entity do the work of any portion of a graded assignment for them, which includes purchasing work from a company, hiring a person or company to complete an assignment or exam, **and/or fully/partially copying code from other resources (such as Internet) or generating code using generative AI tools (such as ChatGPT)**. Except parts provided by us or the book, if any, all submitted code must be written by *you* and must not be shared. You are not allowed to go over your classmates' or friends' code, and they are not allowed to see your code. You are only allowed to have high-level verbal discussions about the project with other students. In addition, you are not allowed to publish your code partially (in forums or other sites for asking questions) or fully (in public version control services such as GitHub, etc.). **You cannot post your code on your public GitHub account, unless you make it private!** You are also not allowed to share your code with future students of this course. **You may NOT use Chegg.com, CourseHero.com, MyAssignmentHelp.com, or any other sites to post your assignment/code or receive any help.** Not complying with these collaboration and code sharing rules will put you and those with whom you share your code under the risk of plagiarism for this or the following semesters. Cheating/copying of assignments (including Internet resources) will be reported to the dean's office for plagiarism and a grade of F will be recorded for the course. **Just check with the instructor in advance if you are not sure if a certain practice would put you under the risk of plagiarism. A simple MS Teams chat with the instructor will suffice!**

Submission

Submission is through Blackboard, strictly following the instructions below. Your work will be penalized 5 points out of 100 if the submission instructions are not followed.

What to Submit

1. Go source files used to solve the project.
2. `report.pdf` including the following information (write the question first, then your answer underneath):

Question 1: Your name and ID.

Question 2: Did you fully implement the project as described? If not, what parts are not implemented at all, or what parts are not implemented as specified in the project description?

Question 3: For each part, briefly describe how you accomplished the task, and share any details that is asked from you in the project description, such as *screenshots*, *plots*, *etc.*, to prove your work.

Be sure to answer any additional questions in the project description above.

How to Submit

1. Create a directory and name it as the combination of the project number and your UofL student ID number. For example, if a student's ID is 1234567, then the name of the directory will be `ProjectX-1234567` for project X.
2. Put all the files to be submitted (only the ones asked in the *What to Submit* section above) into this directory.
3. Zip the directory. As a result, you will have the file `ProjectX-1234567.zip`.
4. Upload the file `ProjectX-1234567.zip` to Blackboard using the "Attach File" option. You do not need to write anything on the "Submission" and "Comments" sections.
5. You are allowed to make multiple submission attempts but only the last attempt submitted before the deadline will be graded. NO LATE SUBMISSIONS WILL BE GRADED!

1 Grading

Grading of your program will be done by an automated process. Your program will also be passed through a copy-checker program which will examine the source codes if they are original or copied; we have a database of similar solutions from previous years as well as from the Internet. We will also examine your source file(s) manually.

Updates

- No updates.